ANALYZING THE ROLE OF AI-ASSISTED PROGRAMMING IN MODERN SOFTWARE DEVELOPMENT

Advanced Programming Concepts

Assessment: AI-Assisted Programming

Student Name: Dhanushi Dewmindi Panamulla Arachchi

Student ID: 2023s20399

Module: Advanced Programming Concepts

Contents

1. Introduction

Software development has evolved significantly with the introduction of Artificial Intelligence (AI). Modern AI-powered tools assist programmers in generating code, debugging applications, improving software quality, and automating repetitive tasks. These tools have become increasingly popular among software engineers because they improve productivity and reduce development time.

AI-assisted programming combines human expertise with machine intelligence to support developers throughout the software development lifecycle. Rather than replacing programmers, AI acts as a supportive assistant capable of generating suggestions, explaining concepts, identifying errors, and improving code quality.

This assignment investigates the role of AI-assisted programming through the development of a Java-based Library Management System. Two development approaches were explored: a traditional programming approach and an AI-assisted programming approach. The implementations were compared to evaluate the impact of AI on software development efficiency, maintainability, and overall code quality.

2. Conceptual Understanding

2.1 What is AI-Assisted Programming?

AI-Assisted Programming refers to the use of Artificial Intelligence technologies to support software development activities. AI tools assist developers by generating source code, detecting bugs, suggesting optimizations, creating documentation, and improving software quality.

Examples of popular AI-assisted programming tools include:

- ChatGPT
- GitHub Copilot
- Amazon CodeWhisperer
- Tabnine
- Replit AI

These tools use machine learning models trained on large amounts of programming knowledge to provide intelligent suggestions and automate programming tasks.

2.2 Types of AI Coding Tools

Code Generation Tools

These tools generate source code from natural language instructions.

Examples:

- ChatGPT
- GitHub Copilot

Debugging Tools

These tools identify programming errors and provide recommendations for fixing issues.

Examples:

- ChatGPT
- GitHub Copilot

Refactoring Tools

Refactoring tools improve code structure and readability without changing software functionality.

Examples:

- IntelliJ IDEA AI Assistant
- GitHub Copilot

Documentation Tools

These tools automatically generate comments, technical documentation, and explanations.

Examples:

- ChatGPT
- Mintlify

2.3 Advantages of AI-Assisted Programming

The use of AI-assisted programming offers several benefits:

Faster Development

AI can generate boilerplate code quickly, reducing development time.

Improved Productivity

Developers can focus on solving business problems rather than writing repetitive code.

Better Learning Experience

Students can learn programming concepts through AI-generated explanations and examples.

Automated Documentation

AI can generate documentation and comments automatically.

Improved Debugging

AI helps identify errors and suggest corrections.

2.4 Risks and Limitations

Despite its advantages, AI-assisted programming presents several challenges.

Hallucinated Code

AI may generate code that appears correct but contains logical or functional errors.

Security Vulnerabilities

Generated code may include insecure implementations.

Dependency on AI

Over-reliance on AI may reduce problem-solving abilities and programming skills.

Lack of Project Context

AI may not fully understand project-specific requirements or business rules.

Ethical Concerns

Questions regarding ownership, copyright, and accountability can arise when using AI-generated code.

3. Practical Implementation

3.1 Selected Problem

The selected problem for this assignment is a Library Management System developed using Java and Object-Oriented Programming principles.

The system provides the following functionalities:

- Add Book
- Display Books
- Search Book
- Issue Book
- Return Book

3.2 Version A – Traditional Programming Approach

Version A was developed manually without AI assistance.

The implementation consists of three classes:

Book Class

Stores information about books including:

- Book ID
- Title
- Author
- Issue Status

Library Class

Manages the collection of books and performs operations such as:

- Add Book
- Display Books
- Search Book
- Issue Book
- Return Book

Main Class

Provides a console-based menu-driven interface for user interaction.

Features Implemented

- Add Book
- Display Books
- Search Book
- Issue Book
- Return Book

Limitations

- Basic validation
- Limited error handling

- Simpler user interface
- Minimal optimization

Screenshots – Version A

Figure 1 – Main Menu

```
C:\Users\USER\.jdks\cor

1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
```

Figure 2 – Add Book

```
C:\Users\USER\.jdks\corrett

1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
1
Book ID: 1
Title: Pinnocchio
Author: Deduni
```

Figure 3 – Display Books

```
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
2
1 | Pinnocchio | Deduni | Available
2 | Harry Potter | Peenaka | Available
```

Figure 4 – Issue Book

```
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
3
Enter Book ID: 1
Book Issued.
```

Figure 5 – Return Book

```
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
4
Enter Book ID: 2
Invalid Book ID.

1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
4
Enter Book ID: 1
Book Returned.
```

Figure 6 – Search Book

```
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
5
Enter Book ID: 1
1 | Pinnocchio | Deduni | Available
```

3.3 Version B – AI-Assisted Programming Approach

Version B was developed with assistance from ChatGPT.

The AI tool was used to:

- Generate initial code structures

- Improve code readability
- Enhance error handling
- Improve search functionality
- Add validation
- Improve maintainability

Additional Features

Compared to Version A, Version B includes:

- Duplicate Book ID validation
- Improved input validation
- Better search functionality
- Enhanced user messages
- Centralized input handling
- Improved maintainability

Validation Features

- Prevent duplicate book IDs
- Validate numerical input
- Validate empty titles and author names
- Check book availability before issuing
- Check issued status before returning

Benefits of AI Assistance

AI significantly reduced development effort by generating boilerplate code and recommending improvements.

Screenshots – Version B

Figure 7 – Enhanced Main Menu

```
================================
 LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
```

Figure 8 – Duplicate ID Validation

```
================================
 LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
Select an option: 1
Enter Book ID: 2
Enter Title: Use of AI
Enter Author: GPT 4 Mini
A book with this ID already exists.
```

Figure 9 – Search Functionality

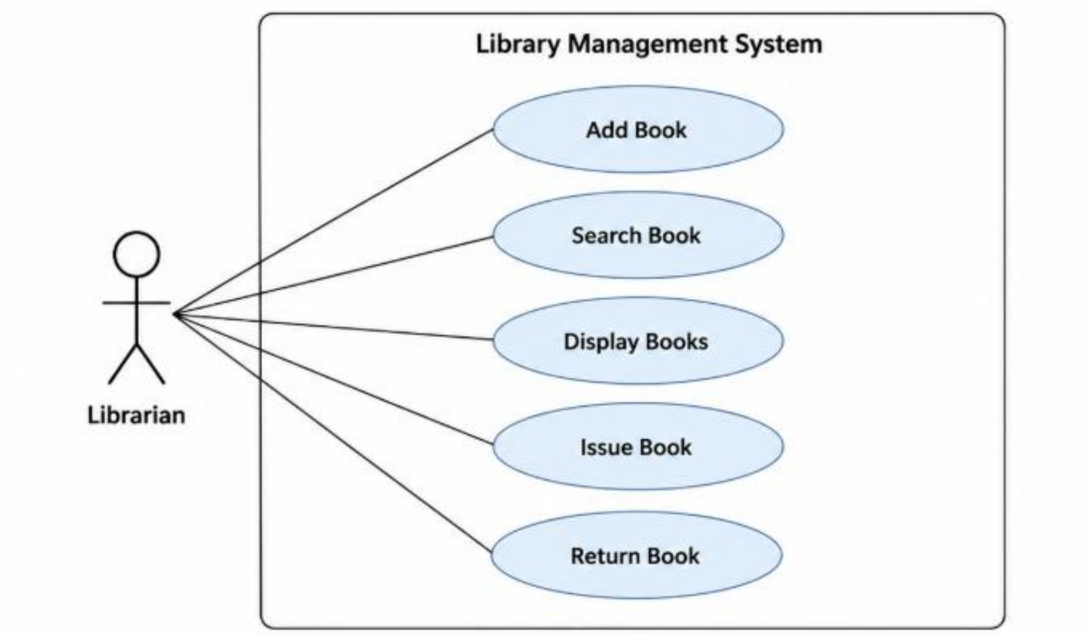
```
================================
 LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
Select an option: 5
Enter Title or Author: Uses of AI
ID: 3 | Title: Uses of AI | Author: Cloude Code | Status: Available
```

Figure 10 – Improved Error Handling

```
================================
 LIBRARY MANAGEMENT SYSTEM
================================
1. Add Book
2. Display Books
3. Issue Book
4. Return Book
5. Search Book
6. Exit
Select an option: 1
Enter Book ID: Use of AI
Please enter a valid number: Please enter a valid number: Please enter a valid number: 2
Enter Title: Use of AI
Enter Author: Me
Book added successfully.
```

3.4 UML Diagrams

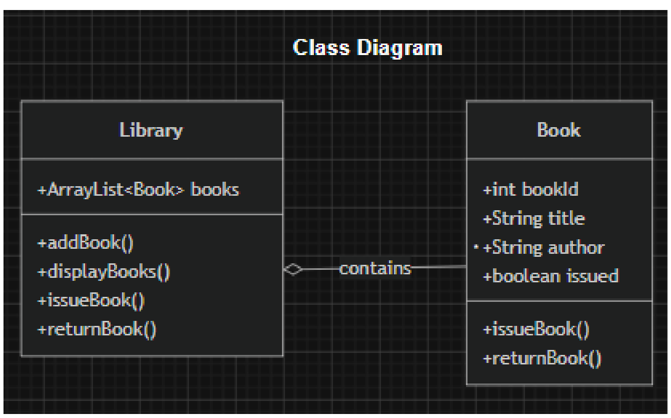
Use Case Diagram



Description:

The Librarian interacts with the system to add books, display books, search books, issue books, and return books.

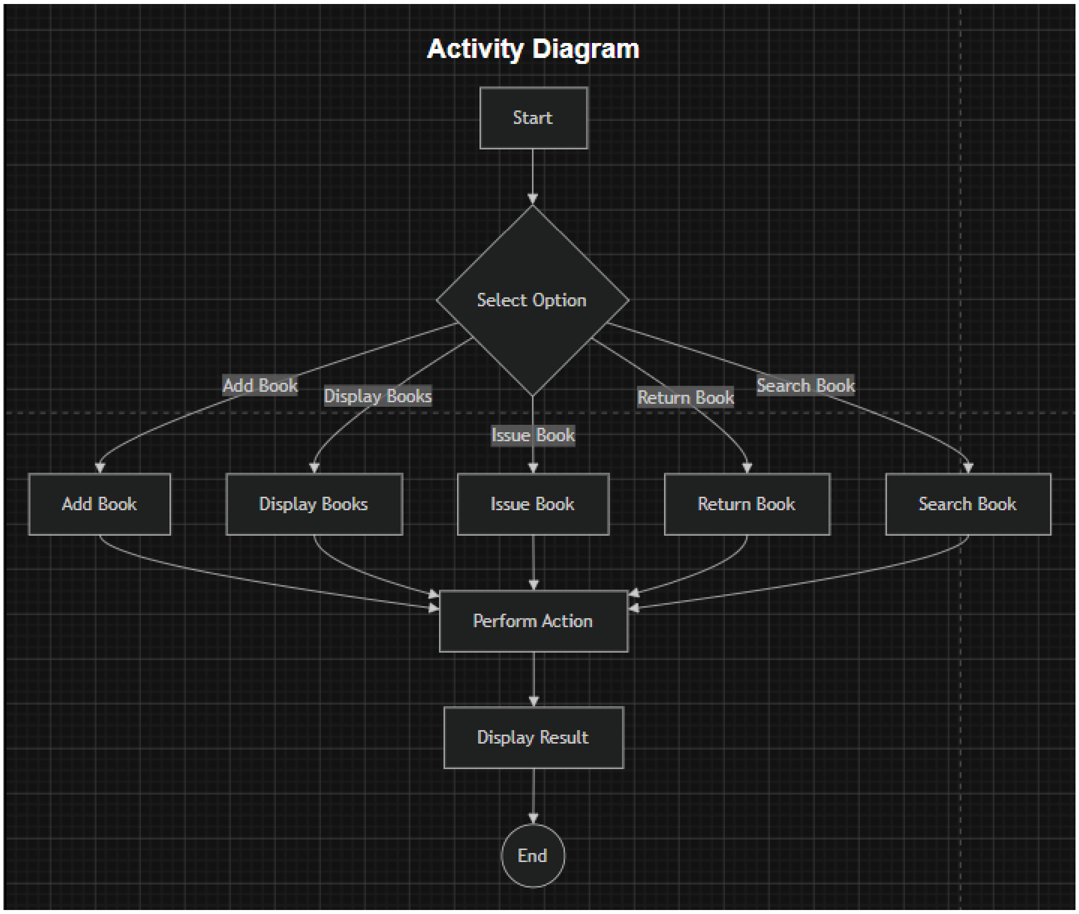
Class Diagram



Description:

The system consists of the Book and Library classes. The Library class manages multiple Book objects using an ArrayList.

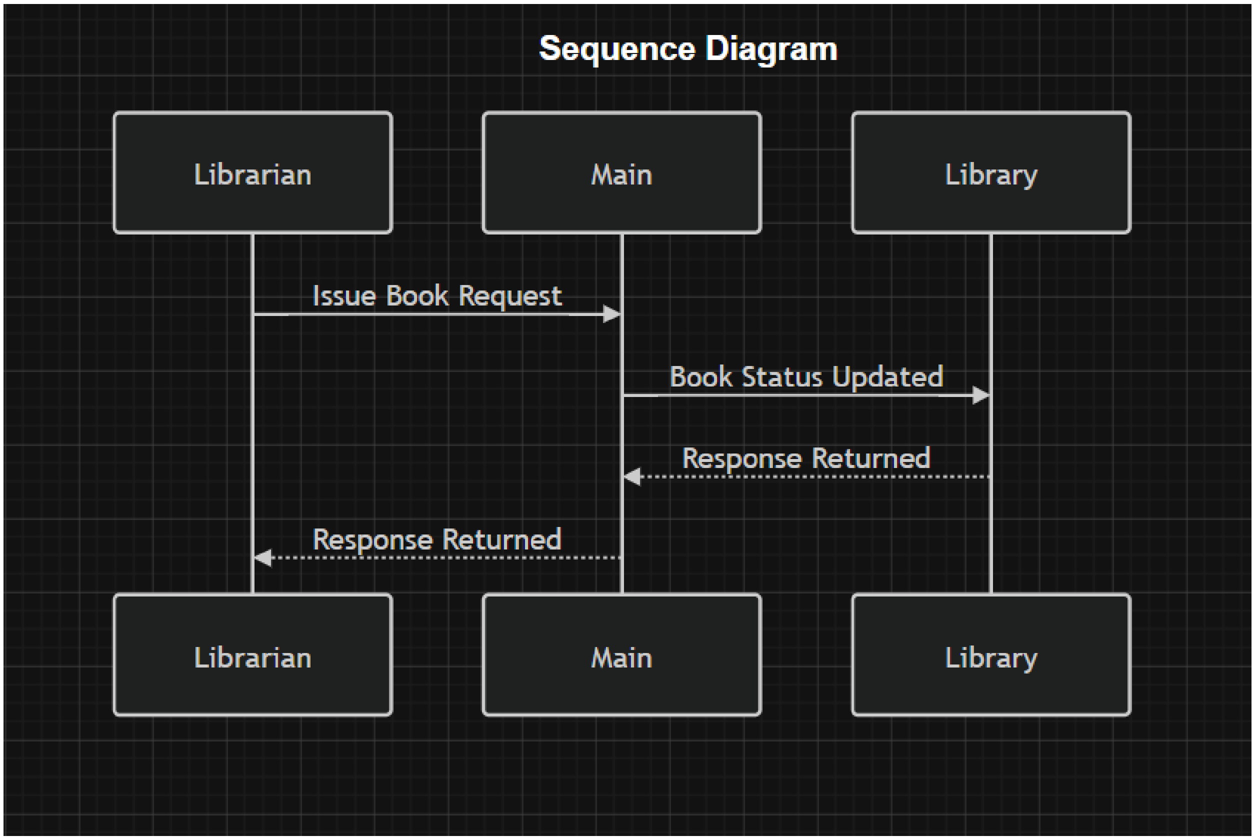
Activity Diagram



Description:

The activity diagram illustrates the workflow from selecting a menu option to performing the requested operation and displaying results.

Sequence Diagram



Description:

The sequence diagram demonstrates the interaction between the user, Main class, Library class, and Book class during system operations.

3.5 Comparison Between Version A and Version B

Criteria	**Version A (Traditional)**	**Version B (AI-Assisted)**
Development Time	Approximately 3 Hours	Approximately 1 Hour
Validation	Basic	Advanced
Error Handling	Limited	Comprehensive
Readability	Moderate	High
Maintainability	Moderate	High
Search Functionality	Basic	Enhanced
User Experience	Basic	Improved

Criteria	Version A (Traditional)	Version B (AI-Assisted)
Productivity	Moderate	High

The AI-assisted implementation demonstrated better maintainability, readability, and overall software quality while requiring significantly less development time.

4. Critical Analysis

4.1 How AI Changed My Coding Approach

AI-assisted programming changed my approach by providing immediate suggestions, code examples, and explanations. Instead of spending considerable time searching for solutions, I was able to obtain assistance directly from the AI tool.

AI also encouraged experimentation with improved implementations and coding practices.

4.2 Did AI Improve or Reduce Understanding?

AI improved my understanding by providing explanations and examples for programming concepts. It acted as a learning companion during development.

However, excessive reliance on AI could reduce understanding if students use generated code without analyzing or testing it.

Therefore, AI should be used responsibly as a learning aid rather than a replacement for programming knowledge.

4.3 Situations Where AI Should Not Be Used

AI-generated code should be used carefully in systems that require extremely high reliability.

Examples include:

- Medical Software
- Aviation Systems
- Banking Systems
- Military Applications
- Safety-Critical Industrial Systems

In such situations, extensive human review and testing are essential.

4.4 Ethical Considerations

Several ethical issues must be considered when using AI-assisted programming.

Accountability

Developers remain responsible for the correctness and security of software regardless of how the code was generated.

Academic Integrity

Students must understand and validate AI-generated code rather than submitting it without analysis.

Copyright and Ownership

Questions regarding ownership of AI-generated code continue to be discussed within the software industry.

Transparency

Developers should clearly identify when AI tools have contributed to software development.

5. Conclusion

Artificial Intelligence is transforming modern software development by improving productivity, reducing development time, and supporting developers throughout the software development lifecycle.

The Library Management System developed in this assignment demonstrated how AI-assisted programming can significantly improve software quality, maintainability, and development efficiency. While AI provides numerous benefits, developers must carefully review generated code and ensure that it meets project requirements.

AI should be considered an intelligent assistant that enhances developer capabilities rather than replacing human knowledge and expertise.

Overall, AI-assisted programming has a positive future in both software engineering education and professional software development when used responsibly and ethically.

6. AI Prompts Used

Prompt 1:

Generate a Java Library Management System using Object-Oriented Programming principles. Include Add Book, Display Books, Search Book, Issue Book, and Return Book functionality.

Prompt 2:

Create a Java Book class with encapsulation, constructors, getters, issueBook(), returnBook(), and toString() methods.

Prompt 3:

Create a Library class using ArrayList to manage books and implement add, search, issue, and return operations.

Prompt 4:

Generate a menu-driven Main class using Scanner and switch-case statements.

Prompt 5:

Improve the Library Management System by adding validation and error handling.

Prompt 6:

Add search functionality that supports searching by title or author.

Prompt 7:

Refactor the code to improve maintainability and readability.

Prompt 8:

Generate UML diagram descriptions for a Library Management System.

Prompt 9:

Compare traditional programming and AI-assisted programming approaches.

Prompt 10:

Discuss ethical considerations related to AI-generated code.

7. GitHub Repository

Repository URL:

https://github.com/danu2003-cy/TI_3003

The repository contains:

- Source Code
- UML Diagrams

- Documentation
- Screenshots
- Commit History

References

OpenAI. ChatGPT. Available at: https://chatgpt.com

GitHub. GitHub Copilot Documentation. Available at: https://github.com/features/copilot

Oracle Corporation. Java Documentation. Available at: https://docs.oracle.com/en/java/

Amazon Web Services. Amazon CodeWhisperer Documentation. Available at: https://aws.amazon.com/codewhisperer/